



PROGRAMAÇÃO I

Aula 07_1 Java

Vinicius Bischoff

O objeto System.out é a saída padrão, que permite exibir as Strings no console (terminal) de comando quando o aplicativo de Java é executado. Dentro desse objeto existem métodos para gerar saídas de Strings, entre elas são: println, print e o printf.

```
public class Texto_sequencia_caractere {  
    public static void main(String[] args) {  
        System.out.print("Antônio \nVieira \ndos\nSanto\n");  
    }  
}
```

System.out - Objeto de saída em Java

Sequência de escape	Descrição
\n	Nova linha. Posiciona o cursor de tela no início da próxima linha
\t	Tabulação horizontal. Move o cursor de tela para a próxima parada de tabulação.
\r	Posiciona o cursor da tela no início da linha atual - não avança para a próxima linha. Qualquer saída de caracteres gerada depois de algum retorno já gerado é sobrescrito os caracteres anteriores gerados na linha atual.
\\	Barras invertidas. Utilizada para imprimir um caractere de barra invertida.
\"	Aspas duplas. Utilizada para imprimir um caractere de aspas duplas. Exemplo, <code>System.out.println("\aspas\");</code> exibe "aspas"

O método printf()

O argumento do método printf é uma **String de formato** que pode consistir em texto fixo e **especificadores de formato**.

A letra “**f**” no final da palavra “print” significa “**formatted**” ou seja exibe os dados formatados.

Os especificadores de formato são como marcadores de lugares para um valor, especificando o tipo da saída dos dados que iniciam com um sinal de porcentagem (%) seguido por um caractere representando seu tipo de dado.

System.out - Objeto de saída em Java

Veja no exemplo, o especificador de formato %s, que é um marcador de lugar para uma String, se for especificado um número no lugar irá gerar um erro.

```
public class Texto_printf {  
    public static void main(String[] args) {  
        System.out.printf("%s\n%s\n", "Antonio", "Vierira");  
    }  
}
```

O método printf()

%d	representa números inteiros
%f	representa números floats
%2f	representa números doubles
%b	representa valores booleanos
%c	representa valores char

System.out - Objeto de saída em Java

System.out.println - Insere uma nova linha, deixando o marcador posicionado na linha abaixo.

System.out.print - Mantém o cursor na mesma linha. Geralmente são utilizadas sequências de escape para pular uma linha.

System.out.printf - Especifica o formato da entrada do tipo de valor, que deve ser o mesmo tipo de dados apontado na instrução. Se possuir alguma dúvida verifique a tabela acima dos tipos de dados que podem ser usados.

Compreendendo a Saída do Objeto

Vamos tentar entender o que acontece quando imprimimos um objeto.

Quando chamamos o método `System.out.print()`, o método `toString()` da classe `Object` é invocado.

Como sabemos, todas as classes em Java estendem a classe `Object`. Portanto, o método `toString()` pode ser aplicado a qualquer instância de qualquer classe.

Este método retorna uma string que é composta pelo nome da classe e o código hash do objeto. Esses dois estão conectados pelo símbolo `@`.

System.out - Objeto de saída em Java

```
J Student.java > Student > main(String[])
1  class Student {
2      private String studentName;
3      private int regNo;
4      private Double gpa;
5
6      Student(String s, int i, Double d) {
7          this.studentName = s;
8          this.regNo = i;
9          this.gpa = d;
10     }
11
12
13
14     public static void main(String[] args) {
15         Student s1 = new Student(s:"Justin", i:101, d:8.81);
16         Student s2 = new Student(s:"Jessica", i:102, d:9.11);
17         System.out.println(s1);
18         System.out.println(s2);
19     }
20 }
```

Observe que a saída não contém as informações dos objetos **s1** e **s2**, mas sim as referências de memória de cada objeto, representadas como uma sequência de caracteres seguida do endereço de memória em hexadecimal. Isso acontece porque a classe **Student** não tem um método **toString()** sobrescrito, o que faz com que o método **toString()** padrão da classe **Object** seja chamado, produzindo a saída acima. Para imprimir as informações dos objetos **s1** e **s2**, seria necessário sobrescrever o método **toString()** na classe **Student**.



```
Student@5acf9800
Student@4617c264
```

A primeira parte da saída mostra o nome da classe (**Student** neste caso) e a segunda parte mostra um código hash exclusivo para o objeto. Obteremos códigos hash diferentes cada vez que executarmos o código acima.

POO – Programação Orientada a Objetos

Como imprimir objetos?

Se quisermos imprimir o objeto e suas propriedades em um formato diferente, precisamos substituir o **método toString()** em nossa classe.

Este método deve retornar uma string. Vamos substituir esse método em nossa classe de aluno e entender o processo.

System.out - Objeto de saída em Java

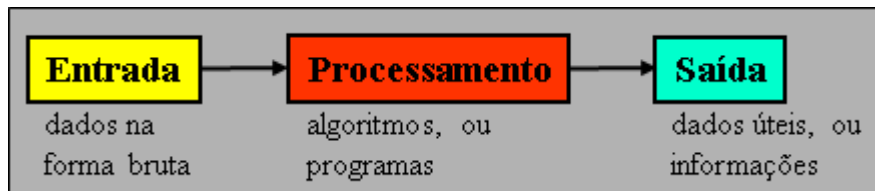
```
J Student.java 3 J StudentToString.java •
J StudentToString.java > StudentToString > StudentToString(String, int, Double)
1 public class StudentToString{
2
3     private String studentName;
4     private int regNo;
5     private Double gpa;
6
7     StudentToString(String s, int i, Double d){
8         this.studentName = s;
9         this.regNo = i;
10        this.gpa = d;
11    }
12    //overriding the toString() method
13    @Override
14    public String toString() {
15        return this.studentName + " " + this.regNo + " " + this.gpa;
16    }
17    Run | Debug
18    public static void main(String[] args) {
19        StudentToString s1 = new StudentToString(s:"Justin", i:101, d:8.81);
20        System.out.print(s1);
21    }
```

Justin 101 8.81

POO – Programação Orientada a Objetos

A classe **Object** é a superclasse de todas as classes em Java. O método **toString()**, que é invocado ao imprimir um objeto, é implementado na classe **Object**. Mas essa implementação não fornece nenhuma informação sobre as propriedades de classe definidas pelo usuário. **Para visualizar adequadamente essas propriedades, precisamos sobrescrever o método **toString()** de nossa classe. Para matrizes, podemos usar diretamente o método **toString()** ou **deepToString()**.**

Processando informações do teclado



Basicamente, todo programa de computador usado na solução de um problema executa algum tipo processamento de dados, que geralmente é desenvolvido em três etapas:

- a. entrada de dados pelo usuário através do teclado;
- b. processamento realizado através de cálculos; e
- c. saídas de informações na tela do computador.

Objeto de Entrada em Java

Classe Scanner

Na linguagem Java, a partir do Java 1.5 ou Java 5.0, o pacote de classes `java.util` disponibilizou a classe `Scanner`, que implementa operações de entrada de dados pelo teclado.

A classe `Scanner` possui vários métodos que possibilitam a entrada de dados de diferentes tipos, entre eles destacam-se:

Objeto de Entrada em Java

- `String next()` - retorna uma cadeia de caracteres simples, ou seja, que não usa o caractere espaço em branco;
- `double nextDouble()` - retorna um número em notação de ponto flutuante normalizada em precisão dupla de 64 bits (usado para receber valores reais ou monetários);
- `boolean hasNextDouble()` - retorna true se o próximo dado de entrada pode ser interpretado como um valor `double`;
- `int nextInt()` - retorna um número inteiro de 32 bits;
- `boolean hasNextInt()` - retorna true se o próximo dado de entrada pode ser interpretado como um valor `int`;
- `String nextLine()` - retorna uma cadeia de caracteres, por exemplo: “Aprendendo java”;
- `long nextLong()` - retorna um número inteiro de 64 b

Objeto de Entrada em Java

Para utilizar a classe Scanner em uma aplicação Java deve-se proceder da seguinte maneira:

1.Importar o pacote java.util:

```
1 | import java.util.Scanner;
```

2.Instanciar e criar um objeto Scanner usando o dispositivo padrão de entrada (System.in):

```
1 | Scanner ler = new Scanner(System.in);
```


Objeto de Entrada em Java

A partir do objeto Scanner, é possível chamar vários métodos para ler diferentes tipos de dados. Por exemplo, o método `nextInt()` lê um número inteiro do usuário:

```
int num = sc.nextInt();
```

O método `nextDouble()` lê um número real:

```
double num = sc.nextDouble();
```

O método `nextLine()` lê uma linha de texto:

```
String texto = sc.nextLine();
```

Objeto de Entrada em Java

```
import java.util.Scanner; // importa a classe Scanner da biblioteca

public class ExemploScanner {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); // cria um objeto Scanner para ler a entrada do usuário
        System.out.print("Digite um número inteiro: "); // exibe uma mensagem para o usuário
        int num = sc.nextInt(); // lê um número inteiro digitado pelo usuário
        System.out.println("Você digitou o número " + num); // exibe o número digitado pelo usuário
        sc.close(); // fecha o objeto Scanner para liberar os recursos
    }
}
```

As estruturas de decisão e iteração em Java são, respectivamente, mecanismos que os programas utilizam para desviar fluxos de controle de um algoritmo e executar comandos repetidas vezes.

A Estrutura Condicional, possibilita a escolha de um grupo de ações e comportamentos a serem executadas, quando determinadas condições são ou não satisfeitas.

A Estrutura Condicional pode ser **Simples** ou **Composta**.

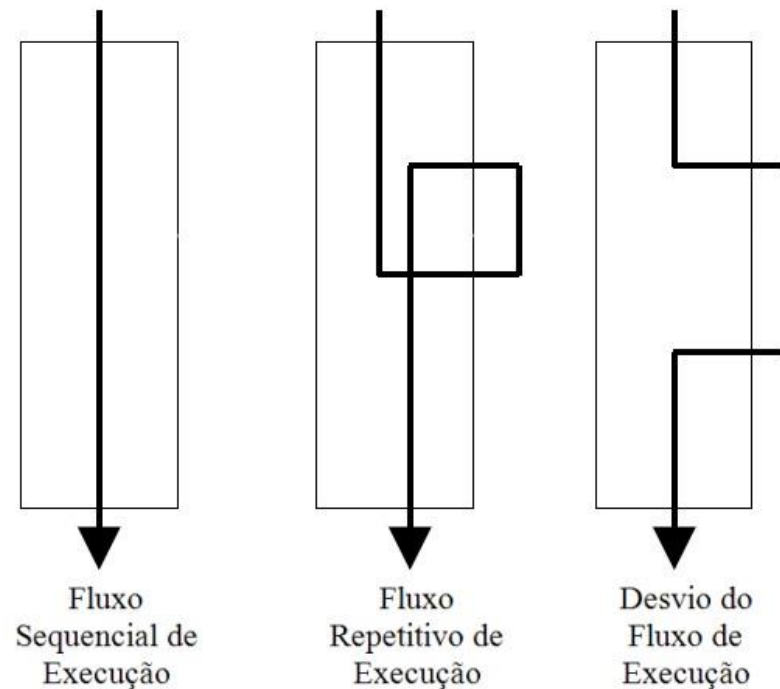
As estruturas de decisão e iteração em Java são, respectivamente, mecanismos que os programas utilizam para desviar fluxos de controle de um algoritmo e executar comandos repetidas vezes.

A Estrutura Condicional, possibilita a escolha de um grupo de ações e comportamentos a serem executadas, quando determinadas condições são ou não satisfeitas.

A Estrutura Condicional pode ser **Simples** ou **Composta**.

Estruturas de decisão e iteração em Java

Em geral, os três tipos de fluxo de execução mais comuns (**sequencial**, **desvio** e **repetição**) encontrados em um algoritmo são mostrados na **Figura 1**, onde os fluxos são representados por setas.



Estruturas de decisão e iteração em Java

Controle de fluxo, é a habilidade de ajustar a maneira como um programa realiza suas tarefas. Por meio de instruções especiais, chamadas de comandos, essas tarefas podem ser executadas seletivamente, repetidamente ou excepcionalmente.

Classificação:

- **Estruturas condicionais:** if-else, switch-case.
- **Estruturas de repetição:** for, while, do-while.
- **Estruturas de exceções:** try-catch-finally, throw.

Estruturas de decisão e iteração em Java

Estrutura if

A estrutura de decisão if é utilizada para impor uma ou mais condições que deverão ser satisfeitas para a execução de uma instrução ou bloco de instruções. A sua forma geral é a seguinte:

A condição sempre irá figurar entre parênteses, após a palavra reservada if, e deve ser uma expressão booleana que resulte em um valor true ou false.

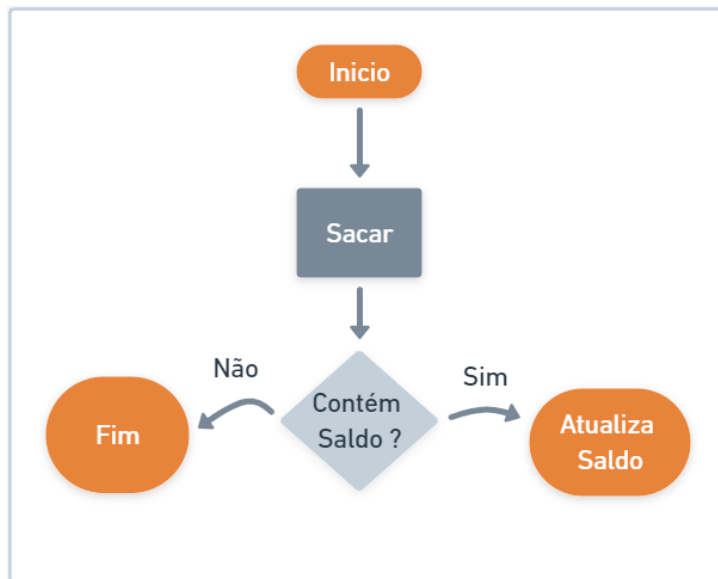
A instrução ou o bloco de instruções somente será executado caso o resultado dessa expressão seja true. Caso o resultado seja false, o fluxo de execução será desviado e a instrução ou o bloco de instruções não será executado.

Havendo uma única instrução condicionada pela estrutura if, ela figura logo após a condição e termina com um ponto e vírgula. A sintaxe é a seguinte:

Estruturas de decisão e iteração em Java

Condicionais Simples

Quando ocorre uma validação de execução de fluxo, somente quando a condição for positiva, consideramos como uma estrutura **Simples**, exemplo:



```
// CaixaEletronico.java
public class CaixaEletronico {
    public static void main(String[] args) {
```

```
        double saldo = 25.0;
        double valorSolicitado = 17.0;
```

```
        if(valorSolicitado < saldo)
            saldo = saldo - valorSolicitado;
```

```
        System.out.println(saldo);
```

```
    }
```

POO – Programação Orientada a Objetos

```
}
```


Estruturas de decisão e iteração em Java

Estrutura if-else

A estrutura de decisão if-else é uma variação da estrutura if. Ela é utilizada para impor uma ou mais condições que deverão ser satisfeitas para a execução de uma instrução ou bloco de instruções e possibilita a definição de uma instrução ou bloco de instruções a serem executados caso as condições não sejam satisfeitas.

A sua forma geral é a seguinte:

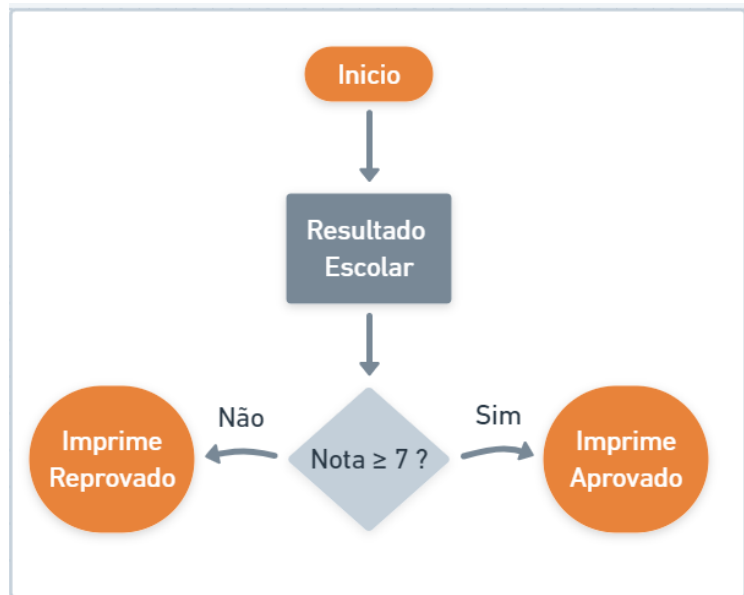
A condição sempre irá figurar entre parênteses, após a palavra reservada if, e deve ser uma expressão booleana que resulte em um valor true ou false.

A primeira instrução ou o bloco de instruções somente será executado caso o resultado dessa expressão seja true. Caso o resultado seja false, o fluxo de execução será desviado e a instrução ou o bloco posterior ao else será executado.

Estruturas de decisão e iteração em Java

Condicionais Composta

Algumas vezes, o nosso programa deverá seguir mais de uma jornada de execução, condicionado a uma regra de negócio, este cenário é denominado **Estrutura Condicional Composta**.



```
// ResultadoEscolar.java
public class ResultadoEscolar {
    public static void main(String[] args) {

        int nota = 6;

        if(nota >= 7)
            System.out.println("Aprovado");

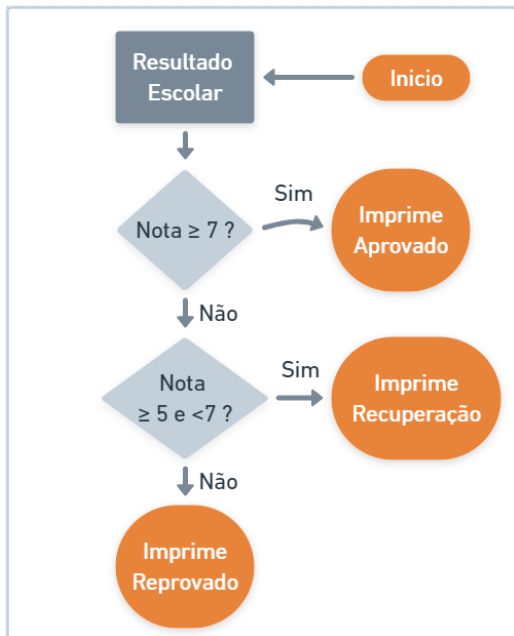
        else
            System.out.println("Reprovado");

    }
}
```

Estruturas de decisão e iteração em Java

Condicionais encadeadas

Em um controle de fluxo condicional, nem sempre nos limitamos ao **se** (if) e **senão** (else), poderemos ter uma terceira, quarta e ou inúmeras condições.



```
// ResultadoEscolarEncadeada.java
public class ResultadoEscolarEncadeada {
    public static void main(String[] args) {
        int nota = 6;

        if (nota >= 7)
            System.out.println("Aprovado");
        else if (nota >= 5 && nota < 7)
            System.out.println("Recuperação");
        else
            System.out.println("Reprovado");
    }
}
```

Estruturas de decisão e iteração em Java

Operador Ternário

Esse código trata-se de um **operador matemático**, com um condicional.

Em alguns casos, ambos podem ter comportamento um pouco diferente.

Lembrando também que o ternário sempre deve retornar valor, e o valor será sempre do mesmo tipo, para ambos os lados da expressão.

Nesse caso, temos duas condições, essas duas condições são iguais, porém a de cima utiliza o operador ternário.

```
1 | (expressão booleana) ? código 1 : código 2;
```

Ao avaliar a expressão booleana, caso ela seja verdadeira, o código 1, declarado após o ponto de interrogação (?) será executado; do contrário, o programa irá executar o código 2, declarado após os dois pontos (:).

POO – Programação Orientada a Objetos

Estruturas de decisão e iteração em Java

Suponha que você está desenvolvendo um software para controle de estoque que precisa informar como está a quantidade de itens de cada produto: se suficiente, para quantidades superiores a 100; em alerta, para quantidades entre 100 e 50; e abaixo do ideal, para quantidades menores do que 50. Como programar esse código?

```
1  int estoque = //valor recuperado do sistema
2
3  if (estoque >= 100) {
4      System.out.println("Produto com quantidade suficiente.");
5  } else if (estoque < 100 && estoque > 50) {
6      System.out.println("Alerta: Avaliar possibilidade de novo pedido.");
7  } else {
8      System.out.println("ATENÇÃO! Faça um novo pedido.");
9  }
```

Estruturas de decisão e iteração em Java

Agora, considere que você precisa de uma estrutura de decisão mais simples, apenas para indicar se estamos na primeira ou na segunda quinzena de um mês. Exemplo de uso do operador ternário:

```
1 | int numeroDias = //valor entre 1 e 30
2 | System.out.println((numeroDias <= 15) ? "Primeira quinzena" : "Segunda quinzena");
```

Estruturas de decisão e iteração em Java

Exemplos:// Cenário 1

```
public class ResultadoEscolarCenario1 {  
    public static void main(String[] args) {  
        int nota = 7;  
        String resultado = nota >=7 ? "Aprovado" : "Reprovado";  
        System.out.println(resultado);  
    }  
}
```

Estruturas de decisão e iteração em Java

// Cenário 2

```
public class ResultadoEscolarCenario2 {  
    public static void main(String[] args) {  
        int nota = 6;  
        String resultado = nota >= 7 ? "Aprovado" : nota >= 5 && nota < 7 ? "Recuperação" : "Reprovado";  
        System.out.println(resultado);  
    }  
}
```


SWITCH

A estrutura de decisão switch-case, ou simplesmente switch, é uma forma simples para se definir diversos desvios no código a partir de uma única variável ou expressão. Havendo uma variável com diversos valores possíveis e sendo necessário um tratamento específico para cada um deles, o uso da estrutura if-else se torna confuso e dificulta a leitura do código. Nesse caso, a clareza e a facilidade estão do lado da estrutura switch.

SWITCH

A sintaxe geral da estrutura switch é a seguinte:

Se for utilizada uma expressão, ela deve retornar um tipo de dados compatível com todos os valores especificados através das declarações case.

Por dedução, todas as declarações devem conter valores de um mesmo tipo. Caso esteja utilizada uma variável, seu tipo também deve ser compatível com os valores das declarações.

Cada um dos valores especificados com as declarações case deve ser um valor literal exclusivo. Se houver algum valor duplicado, será gerado um erro no momento em que você tentar compilar seu código.

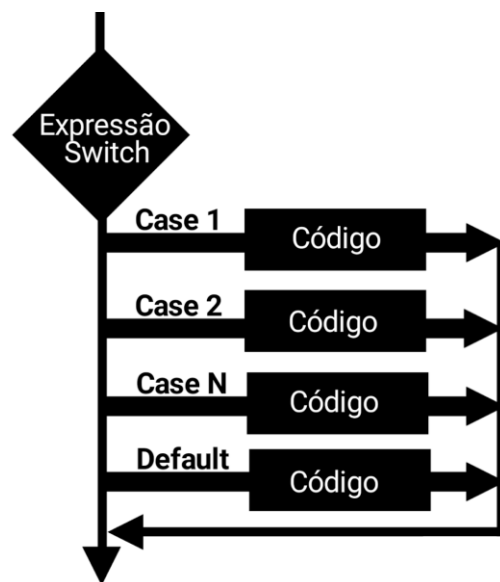
SWITCH

A palavra reservada `break` é utilizada na estrutura `switch` para promover um desvio da execução para a linha posterior ao final de seu bloco.

Geralmente, ele é utilizado como a última instrução de cada declaração `case`. A palavra `default` indica que caso nenhum dos `cases` seja utilizado, a instrução que se encontra no `default` será executada.

Estruturas de decisão e iteração em Java

SWITCH



```
switch (expressão) {  
  
    case valor1:  
  
        // bloco de código que será executado  
  
        break;  
  
    case valor2:  
  
        // bloco de código que será executado  
  
        break;  
  
    case valorN:  
  
        // bloco de código que será executado  
  
        break;  
  
    default:  
  
        // bloco de código que será executado se nenhum dos cases for aceito  
  
}
```

SWITCH

- **switch (expressão)**: é onde passamos a variável de teste que servirá de referência para as comparações que o programa deve fazer;
- **case**: é onde definimos o valor que será comparado com a variável de teste e o código que será executado caso sejam compatíveis;
- **break**: é a declaração opcional de quebra;
- **default**: é a declaração opcional padrão, na qual definimos o código que deve ser executado, se nenhum dos valores declarados nos cases for compatível com o valor passado na variável de teste.

SWITCH

Vamos imaginar que precisaremos imprimir uma medida, com base em mapa de valores, exemplo:

Sigla	Tamanho
P	PEQUENO
M	MÉDIO
G	GRANDE

SWITCH

```
// Modo condicional if/else
public class SistemaMedida {
    public static void main(String[] args) {
        String sigla = "M";

        if(sigla == "P")
            System.out.println("PEQUENO");
        else if(sigla == "M")
            System.out.println("MÉDIO");
        else if(sigla == "G")
            System.out.println("GRANDE");
        else
            System.out.println("INDEFINIDO");

    }
}
```

Estruturas de decisão e iteração em Java

SWITCH

```
// Modo condicional switch / case
public class SistemaMedida {
    public static void main(String[] args) {
        String sigla = "M";

        switch (sigla) {
            case "P":{
                System.out.println("PEQUENO");
                break;
            }
            case "M":{
                System.out.println("MÉDIO");
                break;
            }
            case "G":{
                System.out.println("GRANDE");
                break;
            }
            default:
                System.out.println("INDEFINIDO");
        }
    }
}
```


Equals

O método equals é usado para a comparação. A classe String e as classes Wrapper sobrescrevem o equals para garantir que dois objetos desses tipos, com o mesmo conteúdo, possam ser considerados iguais.

Para descobrir se as referências são iguais, deve-se utilizar o “ == ” para que seja comparado os bits das variáveis.

Toda comparação utilizando o equals irá verificar, primeiro se existe uma sobrescrita do mesmo nas classes comparadas. Caso não haja, o método padrão da classe Object é utilizado.

Equals

O resultado será falso, pois as classes, apesar de terem os mesmos atributos e métodos, a referência é diferente, como eu disse mas acima, para descobrir se as referências são iguais, deve-se utilizar “ == ”.

O equals também poderá ser utilizado para a comparação de Strings:

Nesse caso, a primeira instrução irá gerar uma resposta false, pois não estamos utilizando o método **equalsIgnoreCase**, que ignora as letras maiúsculas e minúsculas, portanto, a segunda instrução irá gerar uma resposta true.

```
1 public class ExemploComparacaoStrings {  
    Run | Debug  
2     public static void main(String[] args) {  
3         String str1 = "hello";  
4         String str2 = "hello";  
5         String str3 = new String(original:"hello");  
6  
7         // Comparando as Strings usando o operador ==  
8         if(str1 == str2) {  
9             System.out.println(x:"str1 e str2 são o mesmo objeto");  
10        } else {  
11            System.out.println(x:"str1 e str2 não são o mesmo objeto");  
12        }  
13  
14        if(str1 == str3) {  
15            System.out.println(x:"str1 e str3 são o mesmo objeto");  
16        } else {  
17            System.out.println(x:"str1 e str3 não são o mesmo objeto");  
18        }  
19  
20        // Comparando as Strings usando o método equals()  
21        if(str1.equals(str2)) {  
22            System.out.println(x:"str1 e str2 possuem o mesmo valor");  
23        } else {  
24            System.out.println(x:"str1 e str2 não possuem o mesmo valor");  
25        }  
    }  
}
```

O primeiro **if** compara **str1** e **str2** usando o operador **==**, que retorna verdadeiro porque ambas as Strings são literais idênticos e, portanto, referenciam o mesmo objeto na memória.

O segundo **if** compara **str1** e **str3** usando o operador **==**, que retorna falso porque **str3** é uma nova instância da String criada com o construtor **new String()**.

Estruturas de decisão e iteração em Java

Em seguida, o código compara as Strings usando o método **equals()**.

O terceiro **if** compara **str1** e **str2** usando o método **equals()**, que retorna verdadeiro porque as duas Strings têm o mesmo valor.

O quarto **if** compara **str1** e **str3** usando o método **equals()**, que também retorna verdadeiro, pois ambas as Strings têm o mesmo valor, mesmo que **str3** seja uma nova instância criada com o construtor **new String()**.

Estruturas de Repetição em Java

Laços de repetição, também conhecidos como laços de iteração ou simplesmente loops, são comandos que permitem iteração de código, ou seja, que comandos presentes no bloco sejam repetidos diversas vezes.

Laços ou repetições são representados pelas seguintes estruturas:

- **For** (para);
- **While** (enquanto);
- **Do While** (faça enquanto).

For

O comando **for** (na tradução literal para a língua portuguesa “para”) permite que uma variável contadora, seja testada e incrementada a cada iteração, sendo essas informações definidas na chamada do comando.

O comando **for** recebe como entrada uma variável contadora, a condição para continuar a execução e o valor de incrementação.

A estrutura de sintaxe do controle de repetição **for** é exibida abaixo:

//estrutura do controle de fluxo for

```
for (bloco de inicialização; expressão booleana de validação; bloco de atualização)
{
    // comando que será executado até que a
    // expressão de validação torne-se falsa
}
```

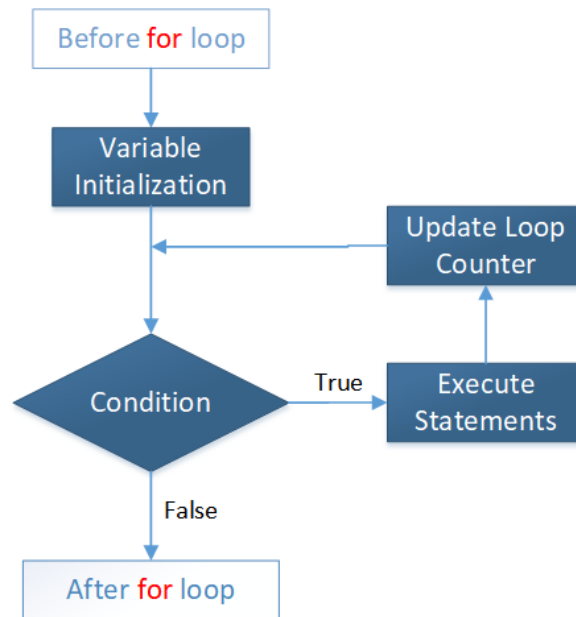

- Inicialização(ões):** A(s) variável(is) é inicializada nesta seção e executada apenas uma vez.
- Condição(ões):** A(s) condição(ões) é(são) definida(s) nesta seção e executada no início do loop sempre.
- Atualização(ões) do contador:** O(s) contador(es) de loop é atualizado nesta seção e executado no final do loop sempre.

Exemplo:

No exemplo abaixo, o programa continua a imprimir a variável chamada *i* do valor 1 a 5. Depois disso, ele sai *do loop for*.

```
public class MinhaClasse {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++)  
            System.out.println(i);  
    }  
}
```

A saída do código acima será: 1 2 3 4 5



For

Vamos imaginar que Joãozinho precisa contar até 20 carneirinhos, para pegar no sono.



1...2...3...

```
public class ExemploFor {  
    public static void main(String[] args) {  
        for(int carneirinhos = 1 ; carneirinhos <=20; carneirinhos ++ ) {  
            System.out.println(carneirinhos + " - Carneirinho(s)");  
        }  
    }  
}
```

Estruturas de Repetição em Java

For position

1. `int carneirinhos = 1;` -> O programa entende que a variável `carneirinhos`, começa com o valor igual a 1 e isso acontece somente uma vez;
2. `carneirinhos <= 20;` -> O programa verifica se a variável `carneirinhos`, ainda é menor que 20;
3. O programa começa a executar o algoritmo, no nosso caso, imprimir a quantidade de carneirinhos em contagem;
4. `carneirinhos ++` -> O programa aumenta em mais 1, o valor da variável `carneirinhos`;
5. O fluxo é finalizado, quando a variável `carneirinhos` for igual a 20.

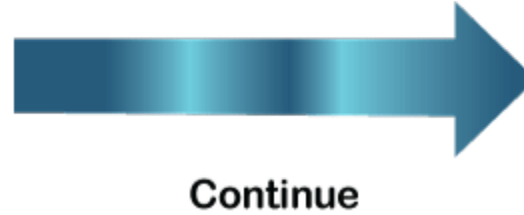
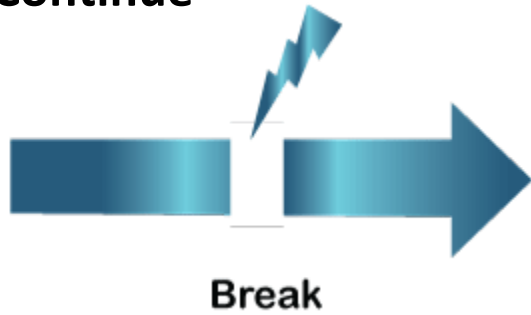
For Each

O uso do **for / each** está fortemente relacionado, com um cenário onde contenha um array ou coleção, e assim, a interação é baseada nos elementos da coleção.

```
public class ExemploForEach {  
    public static void main(String[] args) {  
        String alunos[] = {"FELIPE", "JONAS", "JULIA", "MARCOS"};  
  
        // Forma abreviada  
        for (String aluno : alunos) {  
            System.out.println(aluno);  
        }  
    }  
}
```

- String **aluno : alunos** -> De forma abreviada, é criada uma variável nome obtendo um elemento do vetor a cada ocorrência;
- A impressão de cada aluno é realizada.

Break e Continue



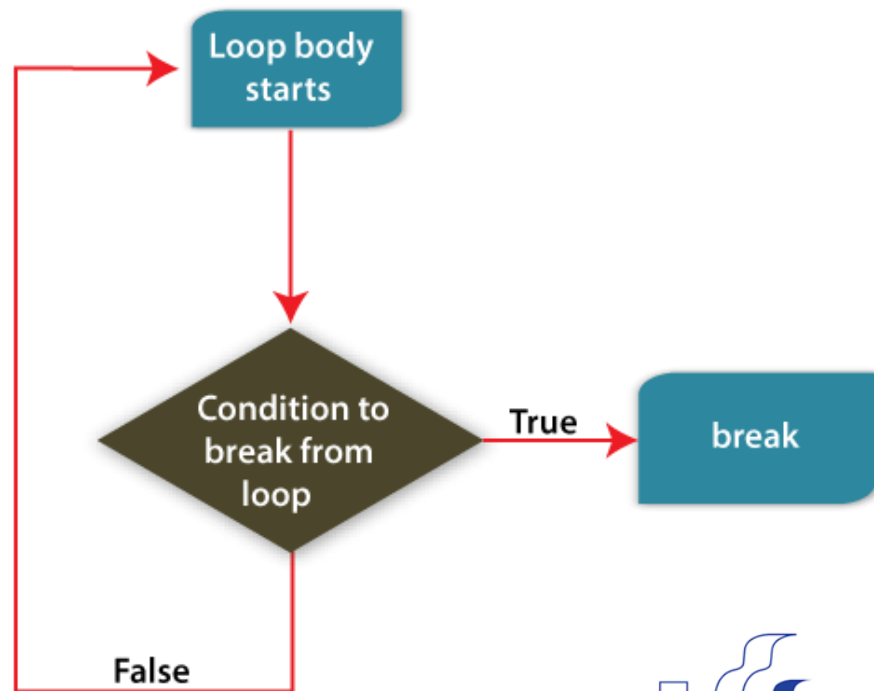
Break significa quebrar, parar, frear, interromper. E é isso que se faz, quando o Java encontra esse comando pela frente.

Continue, como o nome diz, ele 'continua' o laço

Break e Continue

A **instrução break** é usada dentro do loop (for, foreach, while e do-while) e da instrução switch case. À medida que a instrução break era encontrada dentro de um loop, ela imediatamente encerrava a instrução loop e transferia o controle para a próxima instrução, seguida por um loop para retomar a execução. Ele também é usado na instrução de loop aninhado, onde ele quebra o loop interno primeiro e, em seguida, prossegue para quebrar o loop externo. Ele também é usado na instrução switch case para encerrar a execução de um caso específico no programa.

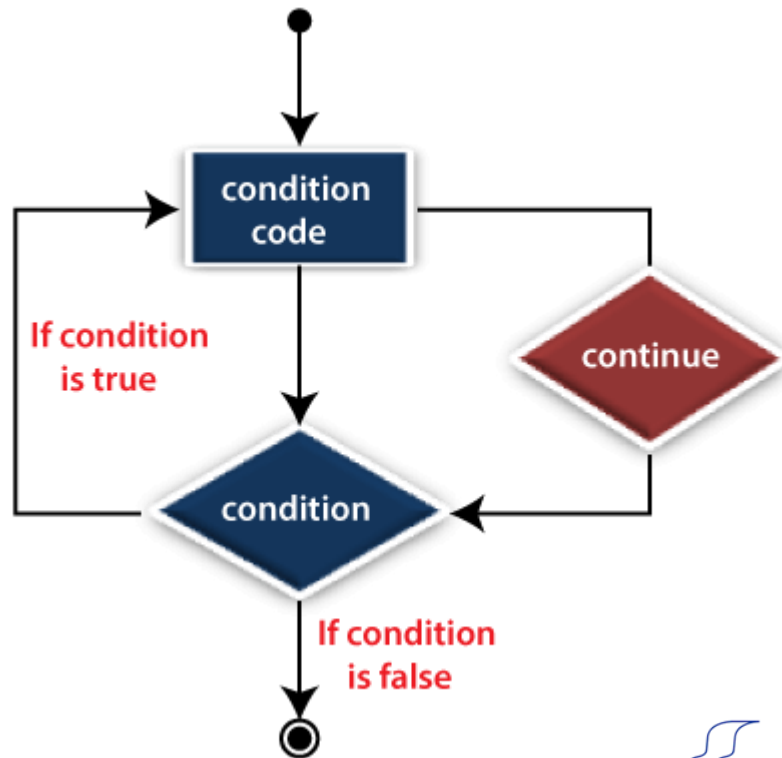
Estruturas de Repetição em Java



Break e Continue

A **instrução continue** é usada no meio do loop **for**, enquanto loop, **loop do-while** e o **loop foreach**. À medida que a instrução **continue** é encontrada em um loop, ela ignora a iteração atual do loop e transfere o controle para o início do loop para execução adicional do loop. Por exemplo, em algumas situações em que queremos ignorar a instrução atual do programa e, em seguida, iniciar a próxima instrução, usamos a instrução **continue**.

Estruturas de Repetição em Java



```
public class ExemploBreakContinue {  
    public static void main(String[] args) {  
  
        // loop for que itera de 1 a 5 (inclusive)  
        for (int numero = 1; numero <= 5; numero++) {  
            // se o valor de numero for igual a 3, interrompe o loop  
            if (numero == 3) {  
                break; // sai do loop for imediatamente  
            }  
            // imprime o valor de numero  
            System.out.println(numero);  
        }  
        // depois do loop, a execução continua aqui  
    }  
}
```

```
public class ExemploBreakContinue {  
    public static void main(String[] args) {  
  
        // loop for que itera de 1 a 5 (inclusive)  
        for (int numero = 1; numero <= 5; numero++) {  
            // se o valor de numero for igual a 3, interrompe o loop  
            if (numero == 3) {  
                continue; // interrompe a iteração atual e passa para a próxima  
            }  
            // imprime o valor de numero  
            System.out.println(numero);  
        }  
        // depois do loop, a execução continua aqui  
    }  
}
```


O laço **while** (na tradução literal para a língua portuguesa “enquanto”) determina que, enquanto uma condição for válida, o bloco de código será executado. O laço **while**, testa a condição antes de executar o código, logo, caso a condição seja inválida no primeiro teste o bloco nem será executado.

A estrutura de sintaxe, do controle de repetição **while** é exibida abaixo:

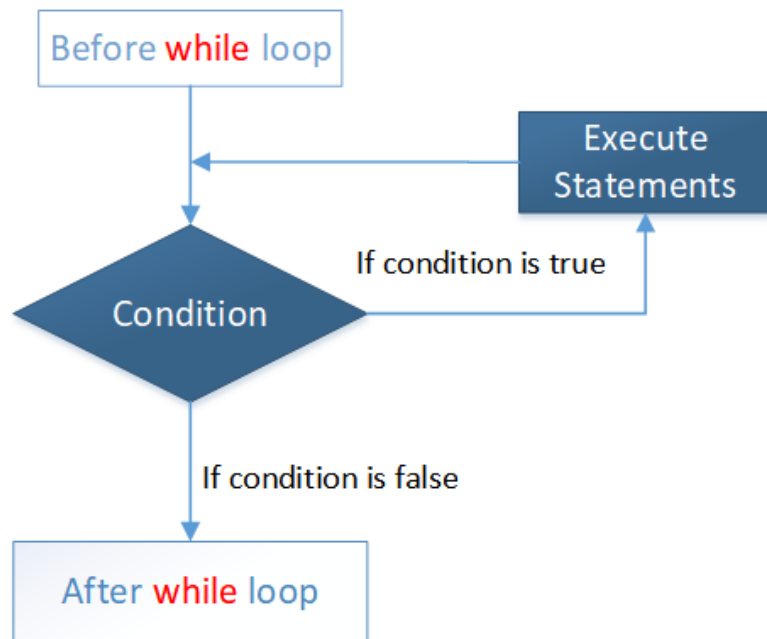
```
while (expressão booleana de validação) {  
    // comando que será executado até que a;  
    // expressão de validação torne-se falsa;  
}
```

While

Estruturas de Repetição em Java

No exemplo abaixo mencionado, o programa usa o loop while para somar todos os inteiros de 1 a 5.

```
public class MinhaClasse {  
    public static void main(String[] args) {  
        int i = 0;  
        int soma = 0;  
  
        while (i <= 5) {  
            soma = soma + i;  
            i++;  
        }  
        System.out.println(soma);  
    }  
}
```



While



Joãozinho recebeu R\$ 50,00 de mesada e foi em uma loja de doces gastar todo o seu dinheiro, logo, enquanto o valor dos doces não igualar a R\$ 50,00 ele foi adicionando itens no carrinho.

POO – Programação Orientada a Objetos

Estruturas de Repetição em Java

```
import java.util.concurrent.ThreadLocalRandom;

public class ExemploWhile {
    public static void main(String[] args) {
        double mesada = 50.0;
        while(mesada > 0) {
            Double valorDoce = valorAleatorio();
            if(valorDoce > mesada)
                valorDoce = mesada;

            System.out.println("Doce do valor: " + valorDoce + " Adicionado no carrinho");
            mesada = mesada - valorDoce;
        }
        System.out.println("Mesada:" + mesada);
        System.out.println("Joãozinho gastou toda a sua mesada em doces");

        /*
         * Não se preocupe quanto a formatação de valores
         * Ire[Sem título]orar os recursos de formatação em breve !!
         */
    } // Este método retorna um número aleatório double entre 2 e 8.
    private static double valorAleatorio() {
        return ThreadLocalRandom.current().nextDouble(2, 8);
    }
}
```

A classe ThreadLocalRandom é usada para gerar números aleatórios.

A chamada "nextDouble(2, 8)" gera um número aleatório double entre 2 (inclusive) e 8 (exclusive), ou seja, pode gerar números entre 2 e 7,999999999999999.

Do While

Estruturas de Repetição em Java

O laço **do / while** (na tradução literal para a língua portuguesa “faça...enquanto”), assim como o laço **while**, considera que, enquanto uma determinada condição for válida, o bloco de código será executado. Entretanto, **do / while** testa a condição após executar o código, sendo assim, mesmo que a condição seja considerada inválida, no primeiro teste o bloco será executado pelo menos uma vez.

A estrutura de sintaxe do controle de repetição **do / while** é exibida abaixo:

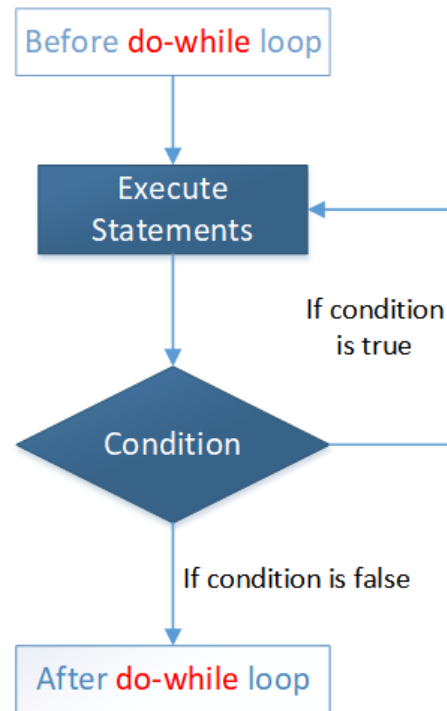
```
do {  
    // comando que será executado até que a;  
    // expressão de validação torne-se falsa;  
  
}  
while (expressão booleana de validação)
```

Do While

Estruturas de Repetição em Java

No exemplo abaixo, mesmo que a condição não seja atendida, o loop **do-while** executa as instruções uma vez.

```
public class MinhaClasse {  
    public static void main(String[] args) {  
        int i = 10;  
        int soma = 0;  
  
        do{  
            soma = soma + i;  
            i= i+1;  
        } while (i <= 5);  
        System.out.println(soma);  
    }  
}
```



While



Joãozinho resolveu ligar para o seu amigo, dizendo que hoje se entupiu de comer docinhos.

retorna o valor booleano negado de "atendeu". Ou seja, se "atendeu" for true, o valor retornado será false e, se "atendeu" for false, o valor retornado será true.

Estruturas de Repetição em Java

```
import java.util.Random;
public class ExemploDoWhile {
    public static void main(String[] args) {
        System.out.println("Discando...");

        do {
            //executando repetidas vezes até alguém atender
            System.out.println("Telefone tocando");

        }while(tocando());

        System.out.println("Alô !!!");
    }
    private static boolean tocando() {
        boolean atendeu = new Random().nextInt(3)==1;
        System.out.println("Atendeu? " + atendeu);
        //negando o ato de continuar tocando
        return ! atendeu;
    }
}
```

A classe Random é usada para gerar números aleatórios.

gerando um número aleatório entre 0 e 2 usando a classe Random, e verificando se o número gerado é igual a 1.

Ao executar o código Java, diferentes erros podem acontecer:

- erros de codificação feitos pelo programador,
- erros devido a entrada errada ou outros imprevistos.

Quando ocorre um erro, o Java normalmente para e gera uma mensagem de erro.

O termo técnico para isso é: Java lançará uma **exceção** (jogará um erro).

De forma interpretativa em Java, um erro é algo irreparável, a aplicação trava ou é encerrada drasticamente.

Já exceções é um fluxo inesperado da nossa aplicação, exemplo:

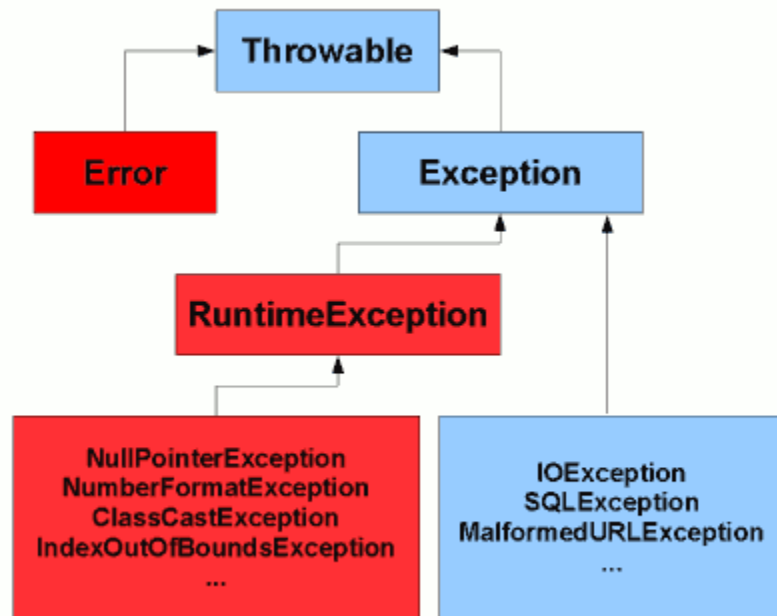
- Querer dividir um valor por zero,
- Querer abrir um arquivo que não existe,
- Abrir uma conexão de banco, com usuário ou senha inválida.

Todos estes cenários e os demais, não são erros mas sim fluxos, não previstos pela aplicação.

É aí que entra mais uma responsabilidade do desenvolvedor, prever situações iguais a estas e realizar o que denominamos de ***Tratamento de Exceções***.

Hierarquia das exceções

A linguagem Java, dispõe de uma variedade de classes, que representam exceções e estas classes, são organizadas em uma hierarquia denominadas **Checked and Unchecked Exceptions** ou *Exceções Checadas e Não Checadas*.



❗ A melhor forma de prever, que pode ocorrer uma exceção, é pensar que ela pode ocorrer.
Lei de Murphy

A linguagem Java, dispõe de uma vasta lista de classes que representam exceções, abaixo iremos apresentar as mais comuns:

Nome	Causa
<code>java.lang.NullPointerException</code>	Quando tentamos obter alguma informação de uma variável nula.
<code>java.lang.ArithmeticException</code>	Quando tentamos dividir um valor por zero.
<code>java.sql.SQLException</code>	Quando existe algum erro, relacionado a interação com banco de dados.
<code>java.io.FileNotFoundException</code>	Quando tentamos ler ou escrever, em um arquivo que não existe.

E quando inevitavelmente, ocorrer uma exceção?

Como nós desenvolvedores podemos ajustar o nosso algoritmo para amenizar o ocorrido?

A instrução **try**, permite que você defina um bloco de código, para ser testado quanto a erros enquanto está sendo executado.

A instrução **catch**, permite definir um bloco de código a ser executado, caso ocorra um erro no bloco try.

A instrução **finally**, permite definir um bloco de código a ser executado, independente de ocorrer um erro ou não.

As palavras-chave try e catch vem em pares:

```
try {  
    // bloco de código conforme esperado  
}  
catch(Exception e) { // precisamos saber qual exceção  
    // bloco de código que captura as exceções que podem acontecer  
    // em caso de um fluxo não previsto  
}
```

❗ O bloco **try** / **catch** pode conter um conjunto de **catchs**, correspondentes a cada exceção **prevista** em uma funcionalidade do programa.

```
public class MyClass {  
    public static void main(String[] args) {  
        try{  
            int x = 10, y = 0, z;  
            z = x/y;  
            System.out.println(z);  
        } catch (Exception e){  
            System.out.println("An error occurred.");  
        }  
    }  
}
```

Composição, agregação e associação em Java

Obrigado.

